All skip-connections can be expanded indifinetly: An Analysis of Representation Dynamics on Vision Models with Differential Geometry

Jaume Ivars Grimalt [1]

Abstract

The abstract must contain a brief intro of the field you are attacking on, the phenomenon you observed, an explanation and an interpretation. Code is available at https://github.com/jaume2000/dynamics_skip_nn.

1. Introduction

Skip Connections-NN (SCNN) wanted to solve grad vanishing problem that VGG introduced. By defining $x = f(x) + x$ it induced some properties: Works in the same feature representation. P.e. A given channel will have the same purpose in the next layer.

(Sander et al., 2022; Queiruga et al., 2020) says that resnets just have the algebraic form of Euler ODE solver, that if params are steps they won'r be ODEs. We apport evidence that residual blocks are 1-step ODE solvers and we therefore have a sum of Ode solvers that are constantly changing of domain and we can accomplish the same dynamics by sharing the weights and we augmentate the resolution of the feature dynamics to study with better quality how X moves along the latent-space manifold in time / depth.

Questions we want to solve:

- Can SCNN's blocks be looped indefinitely? → Yes
- Are all this weights necessary? + Shared networks can reach similar performance? → No + Yes
- What are the dynamics of the features in this two types of models? Statement: The over-parametrized networks undo what the previous layer does, that's why SCNNs and weight-tied are equal. Tangential acceleration is crucial for this affirmation. Here we analyze the geometry of trajectories. Take into account the massive activations here.
- How can we improve discrete models given this evidence? -¿ Delta networks?

[1]Mycrospace AI Research, Valencia, Spain. **AUTHORERR: Missing \icmlcorrespondingauthor.**

arXiv preprint.

2. Related Work

3. Method

We study residual vision networks as a three-part map. Stages 1–2 encode the image onto an input manifold; the longest stage transports a point x_0 on that manifold to a point x_T on an output manifold; stage 4 and the prediction head decode x_T into class logits. We focus on that middle transport—stage 3 in ConvNeXt, Swin and ResNet—where most of the residual depth sits. In ConvNeXt-T, stage 3 has n=9 blocks; we take the trained horizon to be T=n=9 and refine discretisations with repetitions R and step Δt=$1/R$ (Section 3.2).

The question is whether this transport requires n independent residual maps, or whether it is (approximately) the integration of one vector field. We organise the section accordingly:

1. Section 3.1 shows that a weight-tied stage 3 nearly matches an untied ConvNeXt-T, so one shared field already carries most of the accuracy;
2. Section 3.2 tests the Euler expansion over the residual blocks by looping them at inference without fine-tuning, on both weight-tied and standard networks;
3. Section 3.3 compares the geometry of the resulting trajectories (autonomous vs. switched fields);
4. Section 3.4 examines how untied blocks redirect and partially undo each other along depth;
5. Section 3.5 asks whether small per-block corrections on top of a shared field can close the remaining gap.

3.1. Do weight-tied stage-3 networks achieve similar results?

We first ask whether the stage-3 transport needs nine independent blocks, or whether one shared block applied nine times is enough. Sun et al. (2025) study a related question

in transformers: they replace and loop layers with a middle block to test whether all layers implement the same operation. We observe the same phenomenon in ConvNeXt stage 3—ablations that permute, duplicate or substitute blocks are reported in Appendix B—and go one step further: if the stage can be run with a single repeated function, a network trained to use that shared function should reach a similar accuracy level.

We train a vanilla ConvNeXt-T and a weight-tied stage-3 variant under the same recipe (ImageNet-1k, batch size 1024; configuration in Appendix X). Our baseline reaches 80.97% top-1 (28.59M params), slightly below the original 82.1% report—expected under our hardware constraints—but all comparisons below use models trained by us under matched conditions.

The weight-tied model (18.97M params, 33% fewer) reaches 80.36% top-1 (-0.61 points). Sharing all four stages (8.68M params, 70% fewer) reaches 79.01% (-1.96 points).

A network that implements (approximately) the same residual function at every depth can therefore sit close to the untied baseline. That motivates the next subsection: if stage 3 is approximately the Euler discretisation of one vector field, refining that discretisation at inference—looping with step $\Delta t{=}1/R$ while keeping $T{=}9$—should remain meaningful, without fine-tuning. We test this on both the weight-tied network and standard residual models.

3.2. Indefinite looping without fine-tune

Section 3.1 showed that one shared block nearly matches an untied stage 3. We now test whether residual blocks behave as one-step Euler updates of a vector field: if so, repeating each update R times with step $\Delta t{=}1/R$—without fine-tuning—should preserve the trained horizon $T{=}n$ and converge. We apply this probe to weight-tied and standard networks; the geometry of the resulting trajectories is deferred to Section 3.3.

A residual block computes $x + f(x)$ (He et al., 2016). With step size 1, that is exactly one step of the explicit Euler method for the ODE $\dot{x} = f(x, t)$ (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018). Write the residual of block i as

$$f_i(x) \;=\; \mathrm{block}_i(x) - x. \tag{1}$$

Then the native network is one Euler step of size 1 per block. Over a stage of n blocks (e.g. $n{=}9$ in ConvNeXt-T stage 3), starting from the output x_0 of the previous stage,

$$x_{i+1} \;=\; x_i + f_i(x_i), \qquad x_n \;=\; x_0 + \sum_{i=0}^{n-1} f_i(x_i). \tag{2}$$

Some argue that this algebraic form is already enough to call the network an Euler solver (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), but algebraic form alone is not sufficient evidence: Queiruga et al. (2020) argue that ResNets fail to be meaningful dynamical integrators beyond that syntactic similarity, and Sander et al. (2022) show that a ResNet approaches its associated Neural ODE only when the residual maps vary smoothly in depth (otherwise the discrete stack need not be a consistent discretization of a single continuous flow).

We suggest reading the stage as a *sum of several ODEs*, one per block, where the network is trained to solve each unit interval with a single Euler step. The continuous object behind (2) is therefore

$$x_n \;=\; x_0 + \sum_{i=0}^{n-1} \int_i^{i+1} f_i\big(x(t)\big)\, dt, \tag{3}$$

or, block by block,

$$x_{i+1} \;=\; x_i + \int_i^{i+1} f_i\big(x(t)\big)\, dt. \tag{4}$$

Here the sum is the stack of residual blocks, and each integral is the continuous counterpart of the one-step update in (2): instead of taking one Euler step of size 1, we integrate f_i over the unit interval $[i, i+1]$, keeping the horizon of each block equal to 1. For a stage with n blocks trained at unit step, the stage horizon is therefore

$$T \;=\; n \qquad \text{(ConvNeXt-T stage 3: } n{=}9\text{, so } T{=}9\text{).} \tag{5}$$

Repetitions R. Throughout the paper we use a single refinement parameter R (*repetitions*). For a non-tied stage, $R{=}2$ means each of the n blocks is applied twice, for nR residual updates in total. For a weight-tied stage, $R{=}2$ means the *same* shared block is applied $nR{=}18$ times. In both cases we set the Euler step from the trained horizon,

$$\Delta t \;=\; \frac{1}{R} \qquad \Rightarrow \qquad T \;=\; nR \cdot \Delta t \;=\; n, \tag{6}$$

so refining never changes the integration time—only the resolution.

Plain interpolation keeps the field piecewise constant on each block and discretizes the integral with R Euler substeps of size $\Delta t = 1/R$:

$$\int_i^{i+1} f_i\big(x(t)\big)\, dt \;\approx\; \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big). \tag{7}$$

Equivalently, over the whole stage the refined trajectory is

$$x_n \;=\; x_0 + \sum_{i=0}^{n-1} \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big), \qquad \Delta t = \tfrac{1}{R}. \tag{8}$$

When R=1, (8) recovers the original network (2). As R grows with T=n fixed, the scheme approaches the continuous limit (3); we also compare against Midpoint (RK2) and RK4 on the same horizon.

In a weight-tied stage the residual field is the same at every depth, $f_i \equiv f$ for all i. Then (3) collapses to a single autonomous integral over $[0, T]$ with T=n:

$$x_T \;=\; x_0 + \int_0^T f\big(x(t)\big)\, dt. \tag{9}$$

Inference uses nR applications of f with Δt=$1/R$; training uses R=1 (n steps of size 1), hence T=9 for our shared ConvNeXt-T.

3.2.1. Ablations

We apply the refinement (8) (plain interpolation, T conserved via Δt=$1/R$) to our ConvNeXt-T, our weight-tied ConvNeXt-T, Swin-T, ResNet-50 and ResNet-101 (ImageNet-1k validation, top-1). Figure 1 (left) summarises the sweep over R; numbers are in Table 1.

Accuracy decreases monotonically as R grows and plateaus at a continuous limit that matches RK4. Euler (RK1), Midpoint (RK2) and RK4 all converge to the *same* accuracy once the step is fine enough; full Mid/RK4 tables are in Appendix A. At the training step size (Δt=1), a more accurate solver jumps straight to that limit and loses 0.6–0.9 points: the networks are tuned to the error of a single Euler step of size 1.

Wrong step size. Repeating blocks without shrinking the step (e.g. R=10 with Δt=1, so T grows tenfold) collapses every pretrained model to chance ($\approx 0.1\%$–1% top-1). The same holds for the weight-tied network when T leaves the trained horizon (Figure 3 and Table 2). The Euler step must be Δt=$1/R$, equivalently T=n for an n-block stage (T=9 when n=9).

Weight-tied fractional $R < 1$. Table 2 and Figure 2 report the shared network at fixed T=9 with fewer than nine steps ($R < 1$, i.e. $9R$ applications with Δt=$1/R > 1$). Large steps need a higher-order solver: at R=4/9 (4 steps), RK4 reaches 78.9% while Euler falls to 67.8%; Euler with R=2/9 (Δt=4.5) is unstable (5.9%). For $R \geq 7/9$, all three solvers sit within 0.3 points of the trained baseline.

Horizon, not resolution. Varying the number of unit steps at Δt=1 (so T grows) or varying Δt at fixed R=1 (9 steps; T=$9 \cdot \Delta t$) both peak at T=9 (Figure 3): the classification head expects the state reached after that integration time. Integrating too short or too long degrades smoothly near T=9 and collapses far from it. We conclude that the best

Table 1. Plain Euler refinement with Δt=$1/R$ (T conserved). Top-1 (%). For weight-tied, R means $9R$ applications of the shared block (exact $9R$ when available; else nearest measured depth*). ODE limit: RK4 at the finest step in each sweep.

R (Δt)	ConvNeXt	Swin-T	R-50	R-101	Wt-tied
1 (1)	80.95	81.08	80.34	81.67	80.34
2 (0.5)	80.79	80.90	80.06	81.58	80.27
4 (0.25)	80.53	80.65	79.84	81.38	80.22*
10 (0.1)	80.36	80.47	79.61	81.22	80.14*
32 (0.03)	80.30	80.38	79.50	81.12	80.14*
128 (0.008)	80.23	80.34	79.45	81.06	80.12*
ODE limit	80.23	80.32	79.43	81.05	80.12
Gap (R=1 →ODE)	−0.72	−0.76	−0.91	−0.62	−0.22

*Nearest measured shared depth at T=9: 32, 96, 128, 1024 steps respectively (exact $9R$ would be 36, 90, 288, 1152).

Table 2. Shared ConvNeXt-T, fixed horizon T=9 (Δt=$1/R$). Top-1 (%). Trained configuration: R=1 (9 steps of size 1).

R	Steps ($9R$)	Δt	Euler	Mid	RK4
1/9	1	9	47.06	0.61	2.70
2/9	2	4.5	5.85	10.42	47.98
3/9	3	3	37.45	54.47	71.37
4/9	4	2.25	67.83	73.57	78.90
5/9	5	1.8	77.23	78.70	79.97
6/9	6	1.5	79.48	79.85	80.10
7/9	7	1.29	80.19	80.12	80.11
8/9	8	1.13	80.38	80.21	80.11
1	9	1	80.34	80.19	80.11

Euler step is the one that preserves the trained horizon: Δt=$1/R$ for every model (non-tied and weight-tied).

Do refined blocks land on the same x? Near-baseline accuracy does *not* mean that R Euler steps of size $1/R$ reproduce the unit-step state. Figure 4 plots $\|x^{(R)}(t) - x_{R=1}(\lfloor t \rfloor)\|$ for ConvNeXt-T at R=100, Δt=0.01 (massive channel removed from the metric; N=500 images). Two facts stand out.

- **Within each block** the refined trajectory moves *toward* the parallel R=1 checkpoint: after every integer t the distance decays (sawtooth valleys), so the interpolation is not drifting away from the one-step endpoint of that block.
- **Across blocks** the gap grows: mean distance at block ends rises from ≈ 5.6 after the first block to ≈ 76 at t=9, while $\|x\| \approx 203$ at the stage exit (Figure 5), i.e. a relative gap $\|x^{(R)} - x_{R=1}\|/\|x\| \approx 0.36$.

Each block therefore implements a *different* map than its

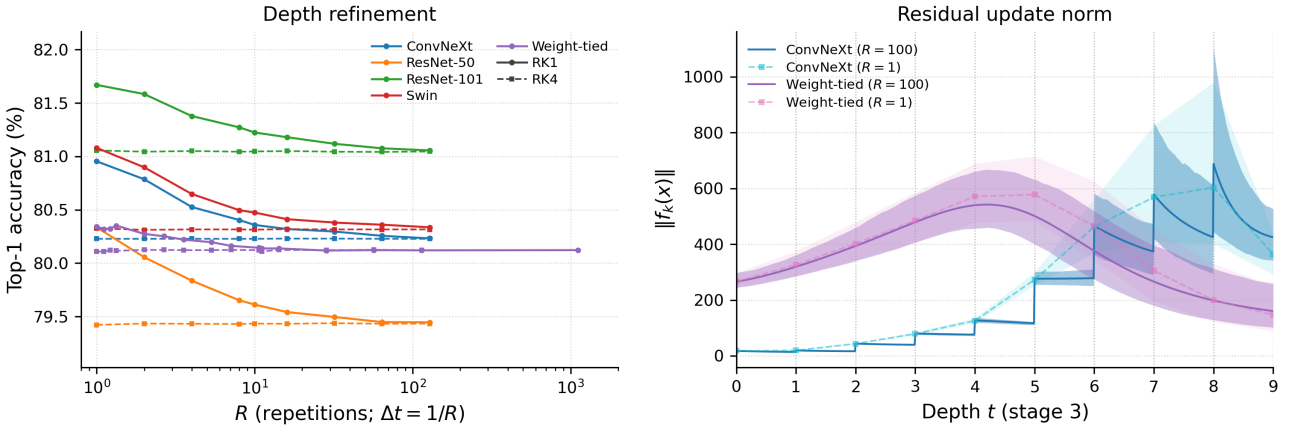


Figure 1. **Left:** top-1 vs. repetitions R (solid: Euler / RK1; dashed: RK4), with $\Delta t{=}1/R$ so $T{=}n$ is conserved (weight-tied: nR applications of the shared block). **Right:** residual-field norm $\|f(x)\|$ along stage 3 (mean and P5–P95 over 500 images). ConvNeXt is piecewise constant in depth; the weight-tied model is a single smooth autonomous field. See Section 3.2.2.

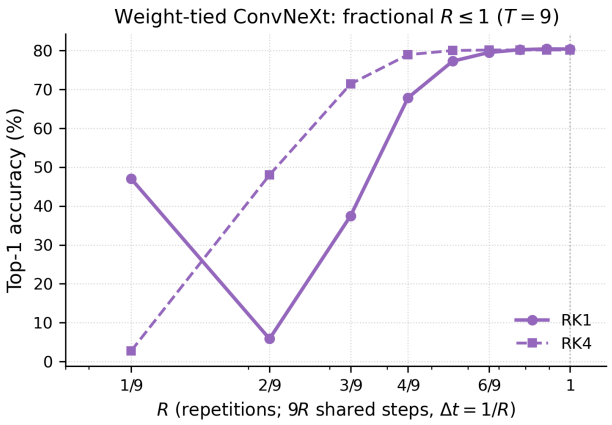


Figure 2. Shared ConvNeXt-T at fixed $T{=}9$ for fractional $R \leq 1$ (RK1 solid, RK4 dashed).

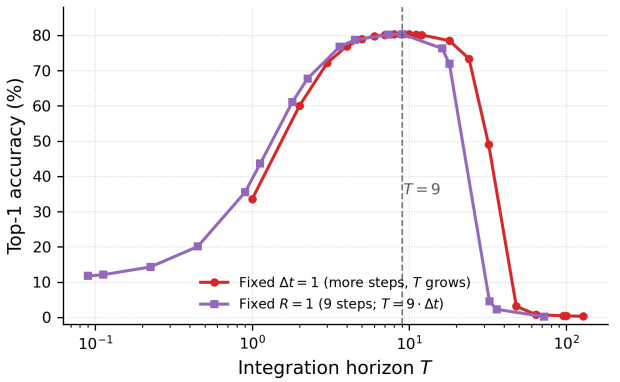


Figure 3. Shared ConvNeXt-T: top-1 vs. horizon T. Red: fixed $\Delta t{=}1$, more steps (T grows). Purple: fixed $R{=}1$ (9 steps), varying Δt ($T{=}9 \cdot \Delta t$). Both peak at the trained horizon $T{=}9$.

one-step counterpart once the input has already drifted; local refinement stays contractive toward the current block's $R{=}1$ target, but errors accumulate through the switched stack and the final x_T is far from the native one.

Finetuning stage 4 and the head after unrolling. As a complementary check, we unroll a pretrained ConvNeXt-T to $R{=}3$ with $\Delta t{=}1/3$ (horizon $T{=}9$ conserved), freeze stem and stages 1–3, and finetune only stage 4 and the classification head. A run of this protocol reached 80.93% top-1—within 0.04 points of our untied baseline (80.97%).[1]

So even though Figure 4 shows that the refined stage-3 endpoint differs substantially from the native x_T in representation space, a light adaptation of the decoder largely restores accuracy when T is kept. The transport changes; the head can re-read it.

Influenced by continuous-in-depth models (Queiruga et al., 2020; Chang et al., 2018), we also tried replacing the piecewise-constant schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by a bilinear blend of consecutive block weights. It performs relatively badly on every architecture (e.g. -6.6 points on ConvNeXt-T at $R{=}128$ vs. plain); full numbers and the residual-norm trajectories are in Appendix C.

[1] Script: `scripts/finetune_s4_r_convnext.py` (default `REPEATS=3`). The experiment directory is not present in this checkout; number to be confirmed when the Leonardo output is synced. For contrast, $R{=}3$ with $\Delta t{=}1$ (broken horizon $T{=}27$) collapses to 41% *without* finetuning (`convnext_r3_quick`).

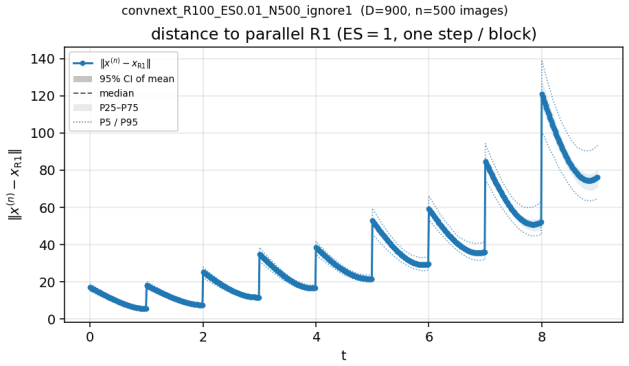


Figure 4. ConvNeXt-T, R=100, Δt=0.01: $\|x^{(R)}(t) - x_{R=1}\|$ (mean over 500 images; massive channel ignored). Sawtooth: approach within each block, jump at every block boundary; valley distance grows with depth (final ≈ 76).

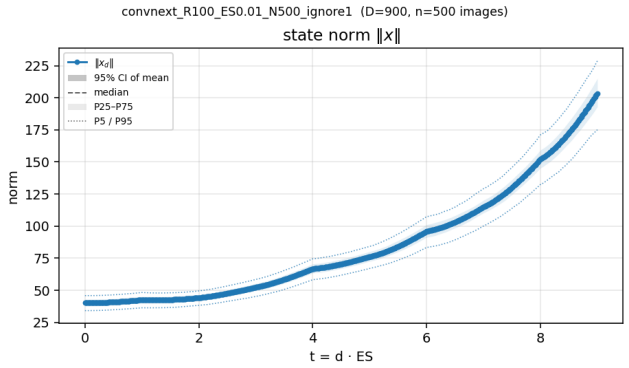


Figure 5. Same run as Figure 4: state norm $\|x\|$ along stage 3 (mean and percentile bands over 500 images; ≈ 203 at t=9).

3.2.2. Piecewise vs. smooth residual fields

Figure 1 (right) makes the dynamical difference between the two regimes concrete. In the non-tied ConvNeXt, $\|f_k(x)\|$ is approximately constant inside each block and jumps at integer depths: a *smooth function by parts*, i.e. an autonomous field on each unit interval $[i, i+1]$ with discontinuous switches of θ between blocks (Equation (3)). In the weight-tied model the same shared f is applied for the whole stage, so the residual norm evolves as one continuous curve that rises and then decays (Equation (9)). Markers at R=1 sit on those curves; refining to R=100 fills in the intra-block path without changing its global shape. This is the geometric counterpart of the accuracy tables: refining a switched stack converges to a slightly worse continuous limit, while refining a shared field stays near the trained accuracy as long as T is held fixed. Sections 3.3 and 3.4 analyse these trajectories in more detail—in particular, how untied blocks redirect the state between integer depths even though each intra-block piece remains integrable.

3.3. Geometrical analysis of feature trajectories

3.3.1. Massive activations excluded from analysis

3.4. Are over-parametrized networks undoing what the previous layers have done?

3.5. Shared field with per-block deltas

4. Results

5. Conclusion

References

Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023.

Chang, B., Meng, L., Haber, E., Tung, F., and Begert, D. Multi-level residual networks from dynamical systems view. In *International Conference on Learning Representations (ICLR)*, 2018.

E, W. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5 (1):1–11, 2017.

Entezari, R., Sedghi, H., Saukh, O., and Neyshabur, B. The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

Haber, E. and Ruthotto, L. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2017.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Lu, Y., Zhong, A., Li, Q., and Dong, B. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *International Conference on Machine Learning (ICML)*, 2018.

Queiruga, A. F., Erichson, N. B., Taylor, D., and Mahoney, M. W. Continuous-in-depth neural networks, 2020. URL https://arxiv.org/abs/2008.02389.

Sander, M. E., Ablin, P., and Peyré, G. Do residual neural networks discretize neural ordinary differential equations? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Sun, Q., Pickett, M., Nain, A. K., and Jones, L. Transformer layers as painters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 25219–25227, 2025.

A. Full Midpoint and RK4 refinement tables

We use the same R (repetitions) as in the main text: $\Delta t{=}1/R$ so that $T{=}n$ is conserved ($n{=}9$ for ConvNeXt stage 3). Tables 3–6 report plain interpolation for Euler (RK1), Midpoint (RK2) and RK4 on each pretrained model. Table 7 gives the same solvers for the weight-tied ConvNeXt-T at fixed horizon $T{=}9$ ($9R$ applications of the shared block). In every case the three integrators converge to the same continuous limit once the step is fine enough; at $R{=}1$, Mid and RK4 already sit on that limit.

B. Block permutation, duplication and substitution

C. Bilinear weight interpolation

Continuous-in-depth work often assumes a smooth path in weight space between consecutive blocks (Queiruga et al., 2020; Chang et al., 2018),

$$\theta(t) \;=\; (1-\alpha)\,\theta_i + \alpha\,\theta_{i+1}, \qquad \alpha = t - i. \tag{10}$$

Replacing the plain schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by (10) hurts every model we tested: at $R{=}128$ the drop vs. plain is between 6.6 and 23.3 points (Table 8). Consecutive blocks are not neuron-aligned, so the midpoint of their weights is not a functional intermediate (Entezari et al., 2022; Ainsworth et al., 2023).

Figure 6 shows the ConvNeXt residual-field norm under plain vs. bilinear schedules at $R{=}100$. Plain keeps the piecewise-constant staircase of Figure 1 (right); bilinear adds high-frequency oscillation between block boundaries while preserving the same global growth of $\|f\|$. Bands are P5–P95 across 500 validation images (same suite as the teaser).

Table 3. ConvNeXt-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.95	80.23	80.23
2	80.79	80.23	80.23
4	80.53	80.23	80.23
10	80.36	80.23	80.23
32	80.30	80.23	80.23
128	80.23	80.22	80.23

Table 4. Swin-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.08	80.29	80.33
2	80.90	80.34	80.31
4	80.65	80.33	80.32
10	80.47	80.32	80.31
32	80.38	80.32	80.32
128	80.34	80.31	80.32

Table 5. ResNet-50, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.34	78.94	79.42
2	80.06	79.41	79.43
4	79.84	79.44	79.43
10	79.61	79.44	79.43
32	79.50	79.43	79.44
128	79.45	79.43	79.43

Table 6. ResNet-101, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.67	80.95	81.06
2	81.58	81.07	81.04
4	81.38	81.06	81.05
10	81.22	81.05	81.05
32	81.12	81.05	81.04
128	81.06	81.04	81.05

Table 7. Shared ConvNeXt-T, fixed T=9 (Δt=$1/R$; $9R$ shared steps), top-1 (%).

R	Steps	Δt	Euler	Mid	RK4
$1/9$	1	9	47.06	0.61	2.70
$2/9$	2	4.5	5.85	10.42	47.98
$3/9$	3	3	37.45	54.47	71.37
$4/9$	4	2.25	67.83	73.57	78.90
$5/9$	5	1.8	77.23	78.70	79.97
$6/9$	6	1.5	79.48	79.85	80.10
$7/9$	7	1.29	80.19	80.12	80.11
$8/9$	8	1.13	80.38	80.21	80.11
1	9	1	80.34	80.19	80.11
$4/3$	12	0.75	80.35	80.14	80.12
2	18	0.5	80.27	80.12	80.12
$32/9$	32	0.28	80.22	80.11	80.12
$128/9$	128	0.07	80.14	80.12	80.12
$1024/9$	1024	0.009	80.12	80.12	80.12
$10000/9$	10000	0.0009	80.12	—	—

Table 8. Bilinear weight interpolation, Euler, top-1 (%).

R	ConvNeXt	ConvNeXt†	Swin-T	R-50	R-101
1	80.96	79.27	81.08	80.34	81.67
2	78.94	72.43	78.93	74.95	72.55
4	76.16	63.13	75.42	70.60	62.42
10	74.42	58.75	73.28	68.58	58.73
128	73.66	57.44	72.14	67.77	57.73
Δ vs plain @128	-6.6	-19.6	-8.2	-11.7	-23.3

†ConvNeXt-T trained without drop path.

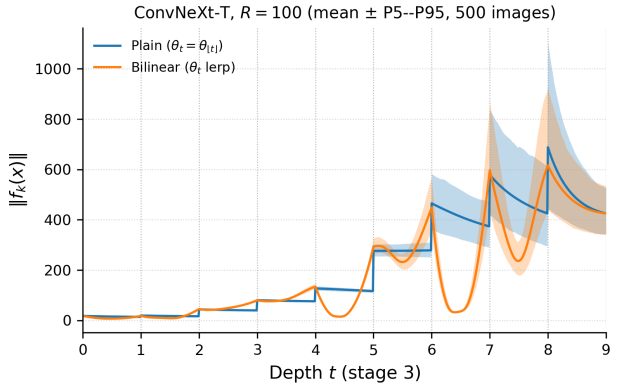


Figure 6. ConvNeXt-T residual norm along stage 3 at R=100: plain vs. bilinear weight interpolation (mean and P5–P95 over 500 images).